

◆ Assignment 1 – Basic Programs

? What is a String in Java?

👉 A **String** is a sequence of characters stored as an object.

? How did you count letters, digits, spaces?

👉 Using:

- `Character.isLetter()`
 - `Character.isDigit()`
 - `Character.isWhitespace()`
-

? What is substring?

👉 It extracts a part of a string using:

```
str.substring(start, end);
```

? How do you reverse a string?

👉 Using loop or:

```
new StringBuilder(str).reverse();
```

? What is an array?

👉 A collection of elements of the same type stored in continuous memory.

? How did you find second highest element?

👉

1. Sort array OR
2. Traverse and track max & second max

? What is matrix transpose?

👉 Rows become columns.

? What is pattern printing?

👉 Using **nested loops** to control rows and columns.

📄 Based on your assignment

◆ Assignment 2 – OOP Concepts

? What is constructor?

👉 A special method used to initialize objects.

? What is constructor overloading?

👉 Multiple constructors with different parameters.

? Why use **this** keyword?

👉 To refer to current object and resolve variable conflicts.

? What is method overloading?

👉 Same method name with different parameters.

? What is encapsulation?

👉 Wrapping data using private variables and public methods.

? What is static variable?

👉 Shared among all objects.

? What is final keyword?

👉 Used to:

- prevent inheritance (class)
 - prevent override (method)
 - make constant (variable)
-

? What is inner class?

👉 A class inside another class.

? What is access specifier?

👉 Controls visibility:

- private
- protected
- public

📄 Based on your assignment

◆ Assignment 3 – Inheritance & OOP Advanced

? What is inheritance?

👉 One class acquiring properties of another.

? What is **super** keyword?

👉 Refers to parent class.

? What is method overriding?

👉 Same method in child class with different implementation.

? What is polymorphism?

👉 One interface, many forms.

? What is runtime polymorphism?

👉 Method call decided at runtime (dynamic binding).

? What is abstraction?

👉 Hiding implementation details using abstract class or interface.

? What is interface?

👉 A blueprint with only abstract methods.

? What is aggregation?

👉 “HAS-A” relationship between classes.

📄 Based on your assignment

◆ Assignment 4 – Collections, Threads, Exception

? Difference between ArrayList and LinkedList?

👉

- ArrayList → fast access
 - LinkedList → fast insertion/deletion
-

? What is HashSet?

👉 Stores **unique elements only** (no duplicates)

? What is LinkedHashSet?

👉 Maintains insertion order + unique elements

? What is thread?

👉 A lightweight process for multitasking.

? What is synchronization?

👉 Controls access when multiple threads use same resource.

? What is exception?

👉 Runtime error that disrupts program flow.

? What is try-catch?

👉 Used to handle exceptions.

? What is custom exception?

👉 User-defined exception class.

? What is file handling?

👉 Reading/writing data using streams.

? What is serialization?

👉 Converting object into byte stream.

📄 Based on your assignment

🔥 Most Important Viva Questions (HIGH CHANCE)

If you remember only these, you're safe:

1. Difference:
 - Overloading vs Overriding
 2. `this` vs `super`
 3. ArrayList vs LinkedList
 4. Abstract class vs Interface
 5. Exception vs Error
 6. Thread vs Process
 7. Encapsulation vs Abstraction
-

◆ Final Tip (important)

If you don't know answer:

👉 Say:

"I'm not fully sure, but I think..."

Teachers like attempt more than silence.

◆ What is a Package in Java?

👉 A package in Java is a collection of related classes and interfaces grouped together.

◆ Real-world example

Think of your laptop folders 

- Folder: **“Projects”**
- Inside:
 - Bank.java
 - Employee.java
 - Student.java

👉 The folder organizes files

➡ Similarly, a **package organizes Java classes**

♦ Why packages are used

- To **organize code**
 - To **avoid name conflicts** (two classes with same name)
 - To provide **access control**
 - To improve **code reusability**
-

♦ Types of packages

✅ 1. Built-in packages

👉 Already provided by Java

Examples:

- `java.util` (ArrayList, Scanner)
 - `java.lang` (String, Math)
-

✅ 2. User-defined packages

👉 Created by programmer

`package mypackage;`

```
class Demo {  
    void show() {  
        System.out.println("Hello");  
    }  
}
```

◆ How to use a package

👉 Import package

```
import java.util.Scanner;
```

👉 Create package

```
package mypackage;
```

◆ Important viva points

- Package is declared at the **top of the file**
 - Helps in **modular programming**
 - Default package exists if you don't specify any
-

◆ Short viva answer



“A package in Java is a namespace that groups related classes and interfaces to organize code and avoid naming conflicts.”

◆ Common viva follow-up questions

? What is default package?

👉 When no package is defined

? What is **import** keyword?

👉 Used to access classes from another package

? Can we access protected members from another package?

👉 Yes, **only through inheritance**

♦ What is a Class (Java)

👉 A class is a blueprint or template used to create objects.

It defines:

- variables (data)
 - methods (behavior)
-

♦ Real-world example

Think of a **Car blueprint** 🚗

- Blueprint defines:
 - color
 - engine
 - speed

👉 But it is not a real car

➡ It's just a design

♦ What is an Object

👉 An object is a real instance of a class.

- If class = blueprint
- Object = actual product

Example:

- Class → Car
 - Objects → BMW, Audi, Tesla
-

♦ Simple Java example

```
class Car {  
    String color;  
  
    void drive() {  
        System.out.println("Car is moving");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Car c1 = new Car(); // object creation  
        c1.color = "Red";  
        c1.drive();  
    }  
}
```

◆ Key differences (important for viva)

Class	Object
Blueprint	Instance
No memory allocated	Memory allocated
Logical entity	Physical entity

◆ One-line viva answer



“A class is a blueprint for creating objects, and an object is an instance of a class containing actual data and behavior.”

◆ Extra (if examiner goes deeper)

? How to create object?

```
ClassName obj = new ClassName();
```

? Where are objects stored?

👉 In heap memory